**ENSI – 2025/2026**
**PHP - MySQL**
**TECHNOLOGIES WEB**
**Date limite de remise : Dimanche 26-Avril-2026 à 23:59**

**PARTIE IV : TRAVAIL A FAIRE**

Dans cette dernière partie de votre projet de cours, votre équipe va améliorer votre site web dynamique en intégrant des scripts côté serveur en PHP avec une base de données MySQL. L'objectif est de traiter les données des formulaires, effectuer des opérations sur la base de données, valider les entrées utilisateur et démontrer un travail collaboratif.

## 1- VALIDATION DE DEUX FORMULAIRES HTML AVEC DES FONCTIONS JAVASCRIPT

Implémentez la validation pour deux formulaires comportant le plus grand nombre de champs de saisie afin de garantir que toutes les entrées utilisateur sont complètes et correctes avant soumission (combiner toute la syntaxe décrite en Annexe §2).

a) Utilisez différents types de validations, tels que :
   - Vérification des champs obligatoires
   - Validations par expressions régulières (par exemple : email, numéro de téléphone ou format de mot de passe)
   - Contraintes de longueur et de plage
   - Validations logiques (par exemple : confirmation de mots de passe identiques)

il me reste empêcher l envoi 4) et le pattern

b) Affichez des messages d'erreur appropriés à proximité des champs concernés afin d'informer les utilisateurs des informations incorrectes ou manquantes avant la soumission du formulaire.

## 2- CONCEPTION ET IMPLÉMENTATION DE LA BASE DE DONNÉES

a) Concevez et implémentez une base de données relationnelle en MySQL composée d'au moins trois tables, chacune représentant un objet JavaScript correspondant utilisé dans la partie 3 de votre projet.
b) Chaque table doit contenir au minimum trois champs, en utilisant des types de données appropriés, des clés primaires et des contraintes reflétant la structure des objets JavaScript. Au moins une table doit inclure un champ représentant une photo.
c) Remplissez chaque table avec au moins cinq enregistrements pertinents et significatifs.
d) Vous devez soumettre à la fois les scripts SQL utilisés pour créer les tables et insérer les données, ainsi que des captures d'écran de phpmyadmin montrant les tables remplies.

## 3- TRAITEMENT DES FORMULAIRES ET RÉCUPÉRATION DES DONNÉES AVEC PHP

a) Modifiez tous les formulaires HTML précédemment développés afin d'envoyer les données soumises vers des scripts PHP correspondants au lieu d'utiliser JavaScript ou d'autres technologies.
b) Chaque script PHP doit traiter les données reçues et les afficher dans un HTML bien formaté (par exemple en utilisant des tableaux ou d'autres éléments sémantiques).
c) Implémentez au moins un script PHP utilisant :
   - Une classe représentant un enregistrement d'une table
   - Un tableau d'objets représentant tous les enregistrements de la table
   - Une fonction permettant de parcourir ce tableau et d'afficher les données sous forme de tableau HTML
d) Tous les autres formulaires doivent interagir avec la base de données MySQL, en récupérant les données pertinentes via des requêtes SQL et en les affichant de manière appropriée.

## 4- ACCÈS AUX DONNÉES VIA DES PAGES WEB PHP [18 POINTS]

Étendez votre projet en créant des pages PHP pour chaque table permettant aux utilisateurs d'interagir avec la base de données à travers des opérations CRUD standards (utiliser tous les types de requête : *query*(), *exec*(), *prepare*(), *execute*() positionnels et nommés) :

a) Fonction de recherche (requête SELECT + utiliser *fetch*(), *fetchAll*() et *fetchObject*()) :
   - Développez une page PHP permettant aux utilisateurs de rechercher dans la base de données selon des critères spécifiques (ex : nom du produit, catégorie, plage de prix).
   - Affichez les résultats correspondants directement sur la même page à l'aide d'un HTML structuré (par exemple des tableaux).
b) Opérations d'insertion, de suppression ou de mise à jour : implémentez la connexion et les trois opérations suivantes avec des éléments HTML appropriés dans des fichiers PHP séparés:
   - Connexion : un fichier PHP permettant de se connecter à votre BD en utilisant PDO
   - Insertion : une page PHP permettant d'ajouter de nouvelles données dans la base (ex : ajouter un nouveau produit)
   - Suppression : une page PHP permettant de supprimer des enregistrements existants (ex : supprimer un produit via son identifiant unique)
   - Mise à jour : une page PHP permettant de modifier des enregistrements existants (ex : mise à jour du prix ou de la disponibilité d'un produit)

Remarque : Même si la validation JavaScript est correcte, les données doivent **toujours être revalidées en PHP avant toute insertion ou modification dans la base de données**.

## 5- TRAVAIL D'ÉQUIPE ET MISE À JOUR DU RAPPORT

- Contribution de l'équipe : tous les membres doivent contribuer de manière équitable. Le travail collaboratif doit être clairement visible.
- Tableau de répartition des tâches : incluez un tableau détaillé (voir annexe §1) précisant pour chaque tâche :
  - le membre responsable,
  - le fichier source concerné,
  - les numéros de lignes correspondants,
  - le type de contribution **(**choisir une seule valeur parmi**) :**
    - **Création** : développement d'un élément à partir de zéro
    - **Implémentation** : ajout d'une fonctionnalité dans une structure existante
    - **Modification** : amélioration ou changement d'un code existant
    - **Correction** : correction d'erreurs (bugs)
    - **Intégration** : liaison entre plusieurs parties (ex : PHP ↔ BD)
    - **Test / Validation** : vérification du bon fonctionnement (tests, validation formulaires)
    - **Documentation** : rédaction du rapport ou commentaires du code
  - une description claire du travail réalisé.
- Soumission du rapport révisé : mettez à jour votre rapport pour inclure :
  - Une description de la conception de la base de données
  - Une explication détaillée du code PHP et de son interaction avec la base de données
  - Des captures d'écran de toutes les nouvelles fonctionnalités et améliorations

## 6- BONUS :

1) Hébergez votre site web sur une plateforme d'hébergement gratuite (par exemple, « **infinityfree** »).
2) Incluez dans le rapport l'URL du site ainsi qu'une capture d'écran de la page d'accueil (index.html).

## 7- Évaluation :

**NB** : Toute tâche non décrite clairement dans le tableau de répartition des tâches ne sera pas notée

### 1. Tableau d'évaluation (50 points)

| Partie | Critère | Pts |
|---|---|---|
| 1. Validation des formulaires JavaScript (8 pts) | Validation de deux formulaires contenant le plus grand nombre de champs | 2 |
| | Utilisation correcte de deux fonctions JavaScript pour valider les champs avant l'envoi au serveur | 2 |
| | Utilisation de différents types de validations à l'aide des deux fonctions JavaScript | 2 |
| | Messages d'erreur clairs et spécifiques pour tous les champs (via alert ou autres éléments HTML) | 2 |
| 2. Conception et implémentation de la base de données (8 pts) | Au moins trois tables bien structurées correspondant à des objets JavaScript | 3 |
| | Utilisation d'au moins 3 attributs avec des types de données et contraintes appropriés pour chaque table | 3 |
| | Au moins 5 enregistrements pertinents par table | 1 |
| | Soumission des scripts SQL et des captures d'écran des tables remplies | 1 |
| 3. Traitement des formulaires et récupération des données avec PHP (10 pts) | Création de pages PHP pour traiter chaque formulaire HTML (envoi uniquement vers des pages PHP) | 2 |
| | Chaque page PHP doit afficher les informations avec des éléments HTML appropriés (tableaux, etc.) | 2 |
| | Définition de classe avec attributs, constructeur, getters, setters et autres méthodes | 2 |
| | Utilisation appropriée d'un tableau d'objets contenant des données complètes et pertinentes | 2 |
| | Utilisation d'une fonction avec des structures d'itération et de sélection pour afficher les données sous forme de tableau | 2 |
| 4. Accès aux données via des pages PHP (14 pts) | Utilisation de requêtes SQL SELECT pour récupérer et afficher les données correctement | 4 |
| | Implémentation d'une fonction de recherche avec affichage des résultats en HTML | 3 |
| | trois types de requêtes SQL différents (INSERT, DELETE, UPDATE) avec éléments HTML appropriés | 5 |
| | Interface conviviale et bonne gestion des entrées utilisateur | 2 |
| 5. Travail d'équipe, et rapport (10 pts) | Contribution équitable et tableau clair de répartition des tâches<br>Rapport mis à jour incluant : description BD, explication PHP, captures d'écran | 10 |
| 6. Bonus | Hébergement du site – site fonctionnel | 5 |

### 2. Présentation Orale

- Chaque équipe doit présenter oralement son projet en présentiel devant un jury composé de deux enseignants. Les détails de la présentation (contenu, date et heure) vous seront communiquer ultérieurement.
- L'enseignant peut poser des questions ciblées sur n'importe quelle partie du projet.
  L'incapacité à expliquer son propre code ou à répondre aux questions pourra entraîner une pénalisation individuelle.

# Annexe

## 1. Exemple de Tableau de répartition des tâches

| Tâche | Membre | Fichier | Lignes | Type | Description |
|---|---|---|---|---|---|
| Formulaire questionnaire | Ali | questionnaire.html | 5–40 | Création | Structure HTML |
| Validation JS | Sara | script.js | 10–60 | Implémentation | Vérification email/password |
| Connexion BD | Yassine | db.php | 1–30 | Intégration | Lien PHP–MySQL |
| Correction bug affichage | Ali | index.php | 45–60 | Correction | Problème boucle |
| Rapport | Sara | rapport.docx | — | Documentation | Partie BD |

## 2. Rappel sur la validation en HTML et JavaScript

### 1. Validation en HTML (validation native)

HTML permet une validation automatique grâce à certains **attributs intégrés** dans les champs de formulaire.

- **Champs obligatoires :** `<input type="text" name="nom" required>`
- **Type de données**
  ```
  <input type="email" name="email">
  <input type="number" name="age">
  ```
- **Contraintes de longueur :** `<input type="text" name="username" minlength="3" maxlength="10">`
- **Contraintes de plage :** `<input type="number" min="18" max="60">`
- **Validation avec expression régulière (pattern) :**
  ```
  <input type="text" pattern="[0-9]{8}" placeholder="8 chiffres">
  ```

👉 Ces validations sont exécutées automatiquement par le navigateur avant la soumission.

### 2. Validation en JavaScript

JavaScript permet de créer des validations **personnalisées et dynamiques**.

| Vérification des champs obligatoires | Validation de longueur |
|---|---|
| `let nom = document.getElementById("nom").value;`<br>`if (nom === "") {`<br>`  alert("Le nom est obligatoire");`<br>`  return false;`<br>`}` | `if (password.length < 8) {`<br>`  alert("Mot de passe trop court");`<br>`  return false;`<br>`}` |
| **Validation avec expression régulière (https://www.w3schools.com/js/js_regexp.asp)** | **Validation logique (confirmation mot de passe)** |
| `let email = document.getElementById("email").value;`<br>`let regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;`<br><br>`if (!regex.test(email)) {`<br>`  alert("Email invalide");`<br>`  return false;`<br>`}` | `if (password !== confirmPassword) {`<br>`  alert("les mots de passe ne correspondent pas");`<br>`  return false;`<br>`}` |

### 3. Affichage des messages d'erreur dans la page

Au lieu d'utiliser `alert`, il est recommandé d'afficher les erreurs près des champs :

```
<span id="erreurNom"></span>
document.getElementById("erreurNom").innerText = "Nom obligatoire";
```

### 4. Empêcher l'envoi du formulaire

La validation est généralement liée à l'événement `onsubmit` :

```
<form onsubmit="return validerFormulaire()">
function validerFormulaire() {
    // vérifications
    return false; // bloque l'envoi si erreur
}
```

### 5. Bonnes pratiques

- Combiner validation HTML + JavaScript
- Afficher des messages d'erreur clairs et spécifiques
- Valider côté client (JS) et côté serveur (PHP)
- Éviter les `alert` excessifs → préférer affichage dans la page